

Compensation of an FIR System

Introduction

In this lab we were given a distorted frequency response of a system and tasked with recovering the original un-distorted response by using a compensation filter. To find the compensation filter, we decomposed the distorted frequency response into two parts: minimum-phase and all-pass. An all-pass frequency response has the property that its magnitude is equal to one for all frequencies, hence its name. It takes the form of:

$$H_{\text{ap}}(z) = \frac{z^{-1} - a^*}{1 - az^{-1}}$$

A minimum-phase function is defined as one where its poles are within the unit circle, and its inverse's poles are inside the unit circle. This means all the poles and zeros of the minimum-phase function are inside the unit circle. Essentially, both a function, $H(z)$, and its inverse, $1/H(z)$, are stable and causal. These types of functions are given the name minimum-phase because they have the property that any non-minimum phase function can be expressed as:

$$\arg[H(e^{j\omega})] = \arg[H_{\text{min}}(e^{j\omega})] + \arg[H_{\text{ap}}(e^{j\omega})].$$

If a system is a minimum system, there will be no H_{ap} function and its phase will be the smallest, i.e. minimum.

If a frequency response function has the property that one of its zeros is outside the unit circle, we can factor the inverse of that zero out of the function, and do algebraic manipulation to decompose that function into minimum-phase and all-pass functions.

Work

The frequency response filter we were given is:

February 11, 2025

$$H_d(z) = (1 - 0.8e^{j0.3\pi} z^{-1}) (1 - 0.8e^{-j0.3\pi} z^{-1}) \\ \times (1 - 1.6e^{j0.9\pi} z^{-1}) (1 - 1.6e^{-j0.9\pi} z^{-1}) .$$

We can easily see that this response has four poles, all at $z=0$. It also has two zeros outside the unit circle, given by $z = 1/(1.6\exp(\pm j0.9\pi))$. Below is the work for finding H_{d_min} , H_{d_ap} , and H_c :

Colin Gregg
ECE 343
Project 2

$$H_d(z) = \left(1 - \frac{4}{5}e^{j0.3\pi} z^{-1}\right) \left(1 - \frac{4}{5}e^{-j0.3\pi} z^{-1}\right) \\ \times \left(1 - \frac{8}{5}e^{j0.9\pi} z^{-1}\right) \left(1 - \frac{8}{5}e^{-j0.9\pi} z^{-1}\right)$$

Factor:

$$\left(1 - \frac{8}{5}e^{j0.9\pi} z^{-1}\right) \left(1 - \frac{8}{5}e^{-j0.9\pi} z^{-1}\right) = \left(\frac{8}{5}e^{j0.9\pi}\right) \left(\frac{8}{5}e^{-j0.9\pi}\right) \left(z^{-1} - \frac{5}{8}e^{-j0.9\pi}\right) \left(z^{-1} - \frac{5}{8}e^{j0.9\pi}\right) \\ = \left(\frac{8}{5}\right)^2 \left(z^{-1} - \frac{5}{8}e^{-j0.9\pi}\right) \left(z^{-1} - \frac{5}{8}e^{j0.9\pi}\right)$$

H_{ap} in form of $\frac{z^{-1} - c^*}{1 - cz^{-1}}$ & $c = \frac{5}{8}e^{\mp j0.9\pi}$.

Multiply & divide by $\frac{1 - cz^{-1}}{1 - cz^{-1}}$ to $H_d(z)$:

$$H_d(z) = \left[\left(1 - \frac{4}{5}e^{j0.3\pi} z^{-1}\right) \left(1 - \frac{4}{5}e^{-j0.3\pi} z^{-1}\right) \right] \left[\left(\frac{8}{5}\right)^2 \times \frac{\left(z^{-1} - \frac{5}{8}e^{-j0.9\pi}\right) \left(z^{-1} - \frac{5}{8}e^{j0.9\pi}\right)}{\left(1 - \frac{5}{8}e^{j0.9\pi} z^{-1}\right) \left(1 - \frac{5}{8}e^{-j0.9\pi} z^{-1}\right)} \right]$$

$\therefore H_{d,min}(z) = \left(\frac{8}{5}\right)^2 \left(1 - \frac{4}{5}e^{j0.3\pi} z^{-1}\right) \left(1 - \frac{4}{5}e^{-j0.3\pi} z^{-1}\right) \left(1 - \frac{5}{8}e^{j0.9\pi} z^{-1}\right) \left(1 - \frac{5}{8}e^{-j0.9\pi} z^{-1}\right)$

Poles: $(0, 0, 0, 0)$

Zeros: $\left(\frac{4}{5}e^{\pm j0.3\pi}, \frac{5}{8}e^{\pm j0.9\pi}\right)$

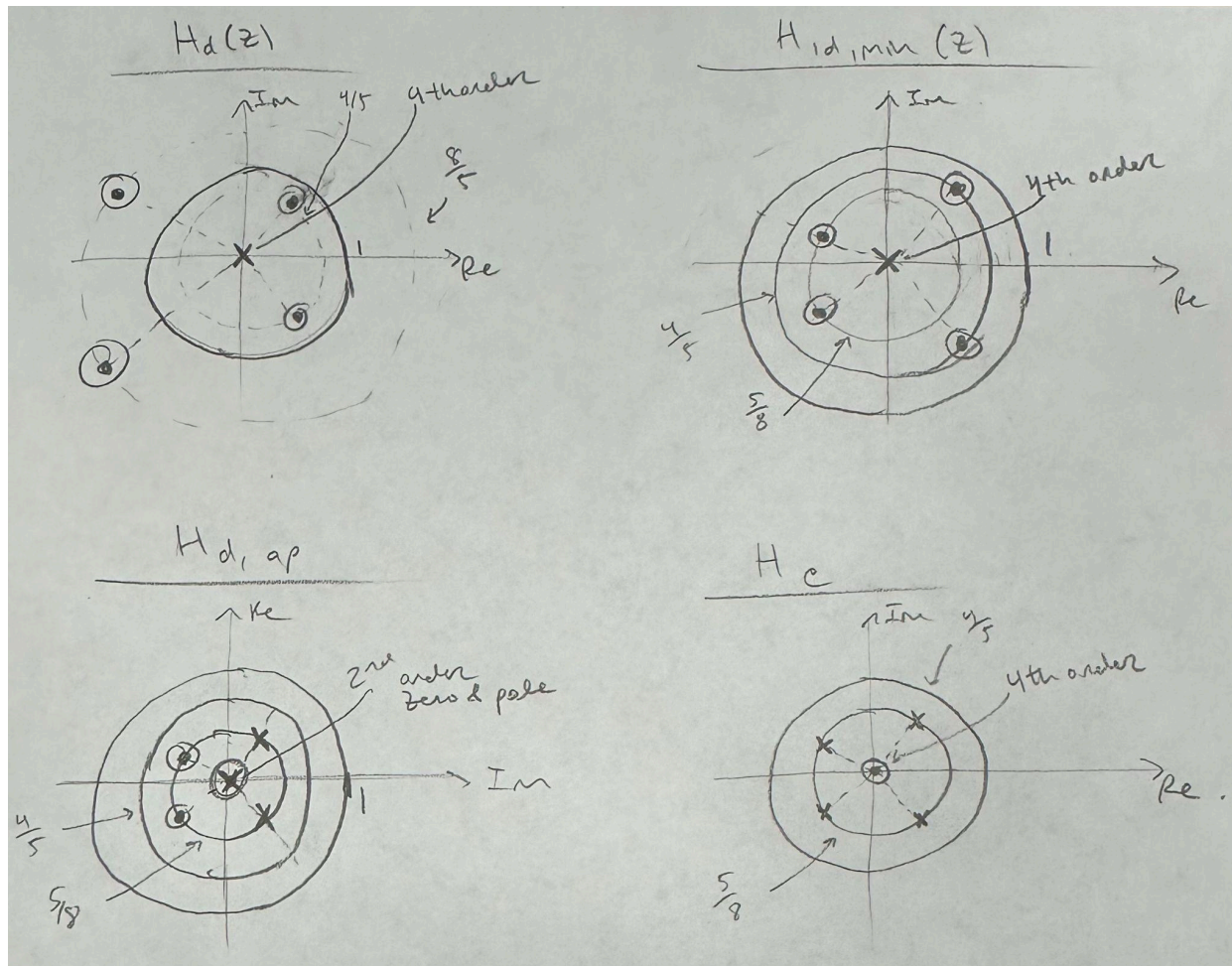
$$H_{d,ap}(z) = \frac{\left(z^{-1} - \frac{5}{8}e^{-j0.9\pi}\right) \left(z^{-1} - \frac{5}{8}e^{j0.9\pi}\right)}{\left(1 - \frac{5}{8}e^{j0.9\pi} z^{-1}\right) \left(1 - \frac{5}{8}e^{-j0.9\pi} z^{-1}\right)}$$

Poles: $(0, 0, \frac{8}{5}e^{\pm j0.9\pi})$

Zeros: $(0, 0, \frac{5}{8}e^{\pm j0.9\pi})$

$$H_c(z) = \frac{1}{H_{d,min}(z)} .$$

The graphs of the poles and zero are as follows:



MATLAB — First Submission

My matlab code is broken down into the following sections:

- Defining frequency response functions
- Graphing

Defining Frequency Response Functions

To start, I defined ω as a variable that varies from 0 to 2π , as that is the domain of figures 5.24-5.26. Then I also defined $z = e^{j\omega}$. I did this because for my script I explicitly declare the functions using the z variable. I tried using `filt()`, `tf()`, `freqz()`, and other similar functions but would continually run into errors with complex coefficients, passing symbols into another function, and having to expand the functions. All-in-all this was the best method I could get to work. Then I defined the zeros and poles as constants to make it easier to declare each

function later. Here is my code so far:

```
% Frequency vector
omega = linspace(0, 2*pi, 1024); % from 0 to 2*pi

% Define z to explicitly write equations
z = exp(1j*omega); % z = e^(jw)

% Coefficients
% for original function
d0 = 1.6*exp(1j*0.9*pi);
d1 = 1.6*exp(-1j*0.9*pi);

% for min & ap funcs
c0 = 0.8*exp(1j*0.3*pi);
c1 = 0.8*exp(-1j*0.3*pi);
c2 = 0.625*exp(1j*0.9*pi);
c3 = 0.625*exp(-1j*0.9*pi);
```

Then I began by defining the following functions:

- Distorted $H_d(z)$
- Minimum-phase $H_{d_min}(z)$
- All-pass $H_{d_ap}(z)$
- Compensating filter $H_c(z)$
- And $H_{mult}(z) = H_c(z) * H_d(z)$,

Where $H_{mult}(z)$ is the frequency response for the distorted signal and the compensating filter in series.

```
% Distorted Frequency Response
H_d = (1-c0*z.^(-1)) .* (1-c1*z.^(-1)) .* (1-d0*z.^(-1)) .* (1-d1*z.^(-1));

% Minimum-phase function
H_d_min = (1.6)^2 * (1-c0*z.^(-1)) .* (1-c1*z.^(-1)) .* (1-c2*z.^(-1)) .* (1-c3*z.^(-1));

% All-pass function
all_pass_numerator = (z.^(-1)-c3) .* (z.^(-1)-c2); % numerator
all_pass_denominator = (1-c2*z.^(-1)) .* (1-c3*z.^(-1)); % denominator
H_d_ap = all_pass_numerator ./ all_pass_denominator; % total ap func

% Compensating filter (H_c(z) = 1 / H_d_min(z))
H_c = 1 ./ H_d_min;

% Multiplication of H_d(z) and H_c(z)
H_mult = H_d .* H_c;
```

Results

We can see some very important traits in these graphs. Firstly, in the log magnitude we see that all-pass and H_{mult} both have a log magnitude of zero. This means the magnitude of the frequency response is uniform and equal to 1, as expected. We can also see that the compensating filter is exactly opposite the distorted signal as they cancel out when they are put in series. Looking at the unwrapped phase plots, we can see that the distorted signal is generally a low pass filter, but with the bumps at the top and bottom. When multiplied with the compensating filter, however, it becomes a much smoother and better looking low pass filter. Looking at the unwrapped phase plots, we can see the same discontinuity jumps as the figures from the textbook. We also can see that the all-pass and H_{mult} has the same frequency response. This, again, checks with our theory that $G(z) = H_{\text{d}}(z) * H_{\text{c}}(z) = H_{\text{d_ap}}(z)$.

For this second submission, I focused on altering my script so that I could use the `grpdelay()` function to calculate the group delay of each signal. In order to do this, I needed to have the expanded polynomial form of each frequency response function. In order to do this I used the `poly()` function. `Poly()` takes in a vector that contains the roots of a function, and returns the coefficients of the expanded polynomial whose roots are the ones given by the input vector. To get the numerator and denominator of the $H_{\text{mult}}(z)$ function I used the `conv()` matlab function. This function returns the convolution of two vectors. If the vectors are coefficients of two polynomial functions, `conv()` simply multiplies the two functions together. The script for this looks like:

```
%{***** Compute Group Delay Using grpdelay *****%}
% Compute numerator polynomial coefficients
b_d = poly([c0, c1, d0, d1]); % Expand H_d(z)
b_c = 1; % H_c(z) is 1/H_d_min(z) so its numerator is 1
b_mult = conv(b_d, b_c); % Multiply numerators

b_d_min = poly([c0, c1, c2, c3]);
b_d_ap = poly([1/c3, 1/c2]);

% Define denominator coefficients (FIR filter, so denominator is just 1)
a_d = 1;
a_c = poly([c0, c1, c2, c3]); % Expand H_c(z);
a_mult = conv(a_d, a_c); % multiply denominators

a_d_min = 1;
a_d_ap = poly([c2, c3]);

% Compute group delay
[gd_d, w] = grpdelay(b_d, a_d, 1024);
[gd_c, ~] = grpdelay(b_c, a_c, 1024);
[gd_mult, ~] = grpdelay(b_mult, a_mult, 1024);

[gd_min_phase, ~] = grpdelay(b_d_min, a_d_min, 1024);
[gd_all_pass, ~] = grpdelay(b_d_ap, a_d_ap, 1024);
%{*****%}
```

Testing Against the Book Example

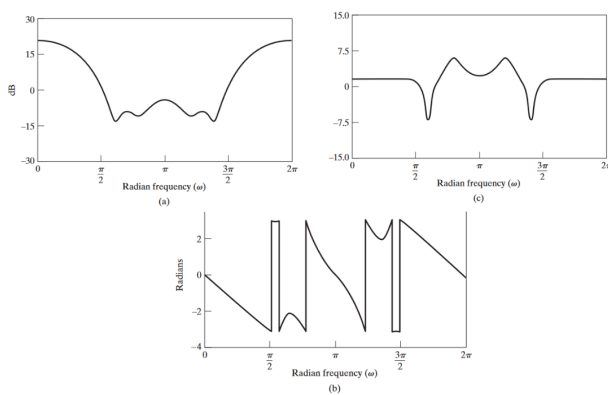
Filter from book example:

$$H_d(z) = (1 - 0.9e^{j0.6\pi} z^{-1})(1 - 0.9e^{-j0.6\pi} z^{-1}) \\ \times (1 - 1.25e^{j0.8\pi} z^{-1})(1 - 1.25e^{-j0.8\pi} z^{-1}).$$

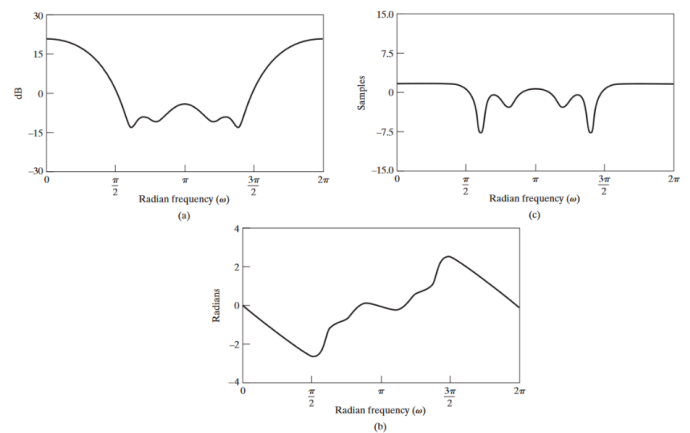
$$H_{\min}(z) = (1.25)^2 (1 - 0.9e^{j0.6\pi} z^{-1})(1 - 0.9e^{-j0.6\pi} z^{-1}) \\ \times (1 - 0.8e^{-j0.8\pi} z^{-1})(1 - 0.8e^{j0.8\pi} z^{-1}),$$

$$H_{\text{ap}}(z) = \frac{(z^{-1} - 0.8e^{-j0.8\pi})(z^{-1} - 0.8e^{j0.8\pi})}{(1 - 0.8e^{j0.8\pi} z^{-1})(1 - 0.8e^{-j0.8\pi} z^{-1})}.$$

And the results given by the book:

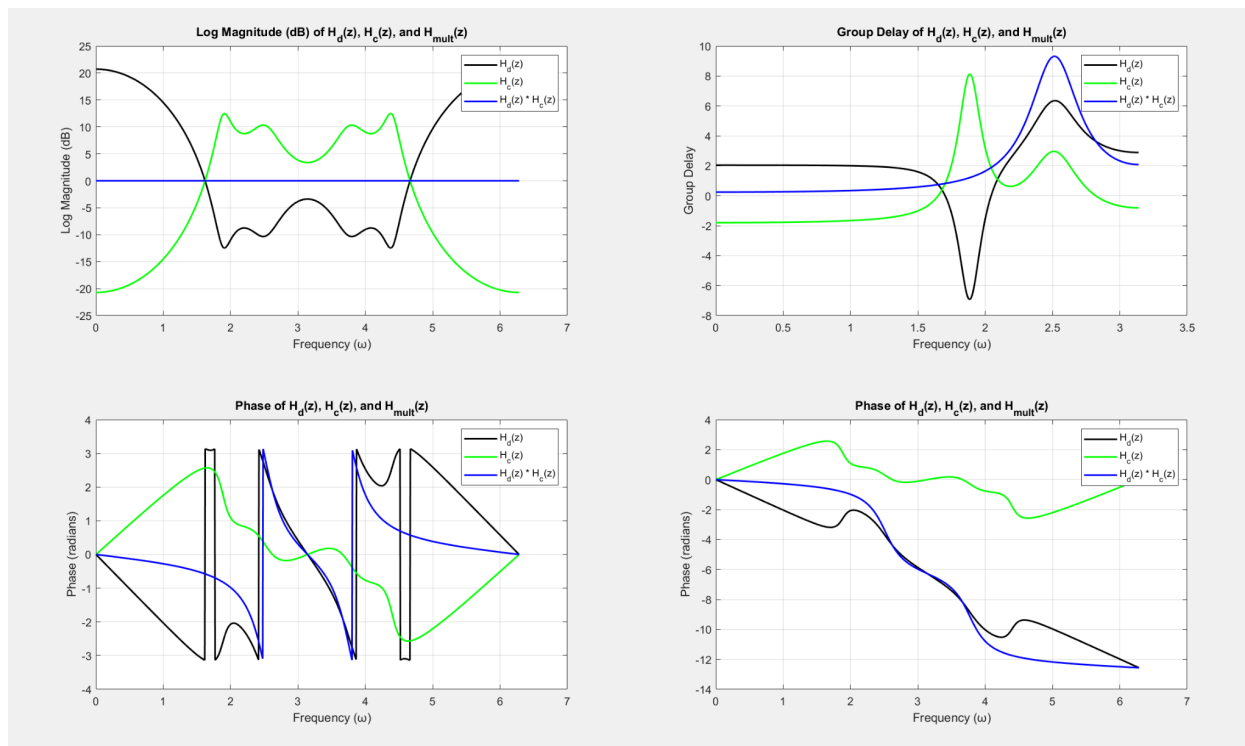


$H_d(z)$

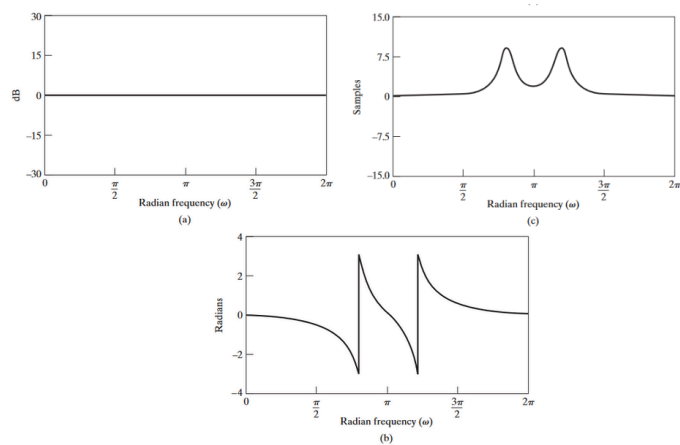


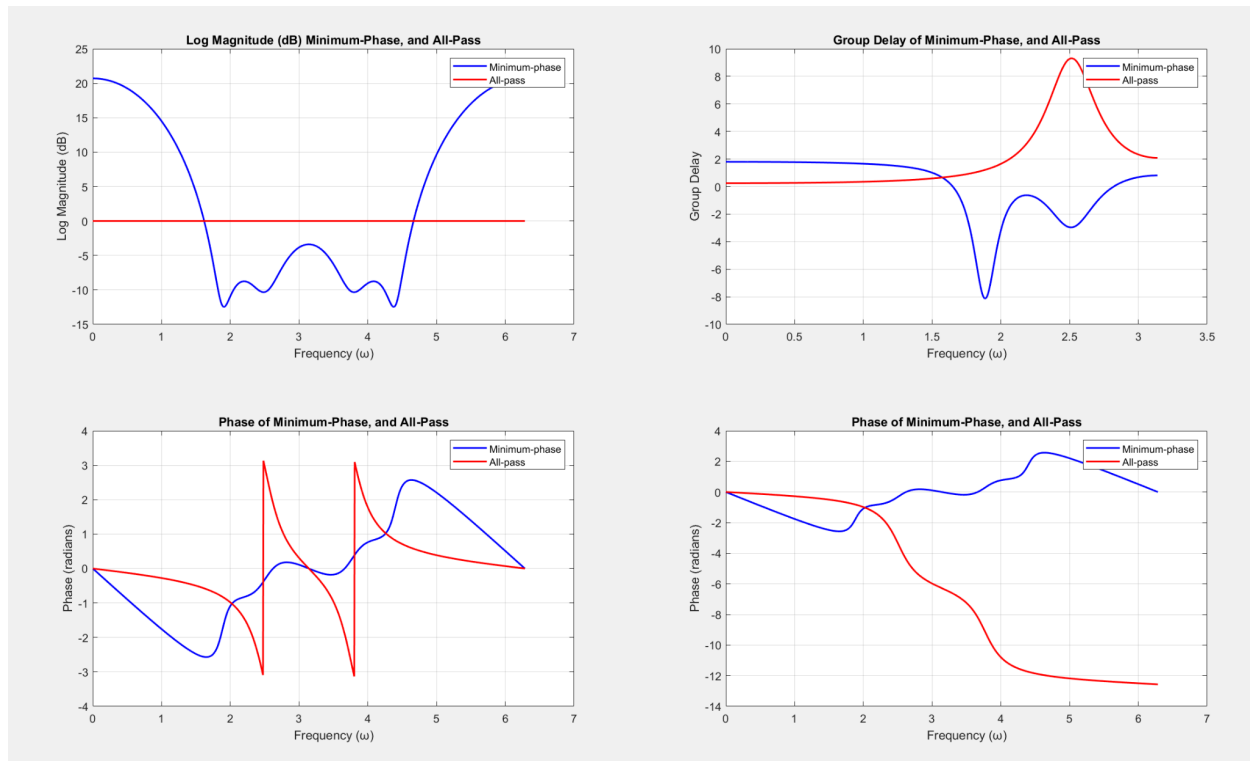
$H_{\min}(z)$

My Results – Book Example, $H_d(z)$, $H_c(z)$, $H_{mult}(z)$:



$H_{d_ap}(z)$:



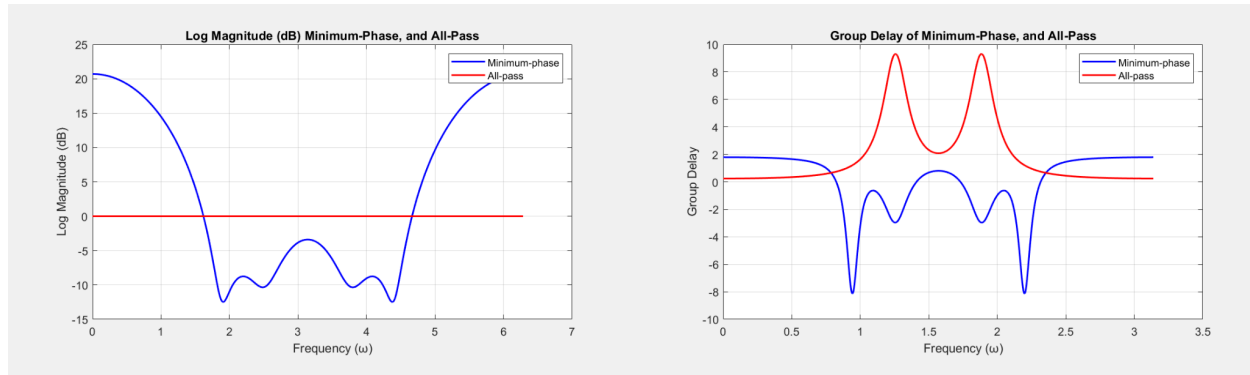


We can see that when I change the zeros and poles for each function to accommodate for the ones given in the book's example, my script exactly recovers the figures from the book. It also uncovered a detail about the book's figures it doesn't say— They use some other method beside `grpdelay()` to calculate the group delay. I notice this because log magnitude and (wrapped) phase are exactly the same as from the book, however, my group delay was quite different. Looking into it, the matlab function `grpdelay()` automatically unwraps the phase of the input function so my group delay is the negative derivative of the unwrapped phase.

The wrapped phase is the angle of the frequency response function constrained to the ranges either $[-\pi, \pi]$ or $[0, 2\pi]$. When the phase goes out of this range it wraps around to the beginning. That is why we see such large discontinuities.

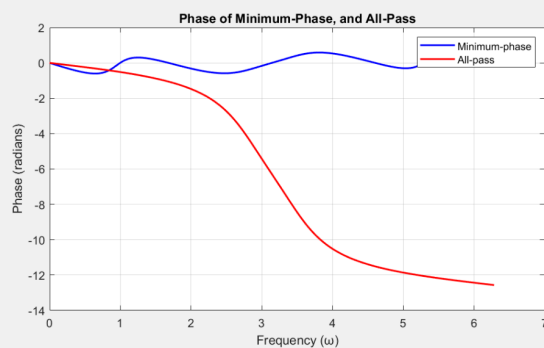
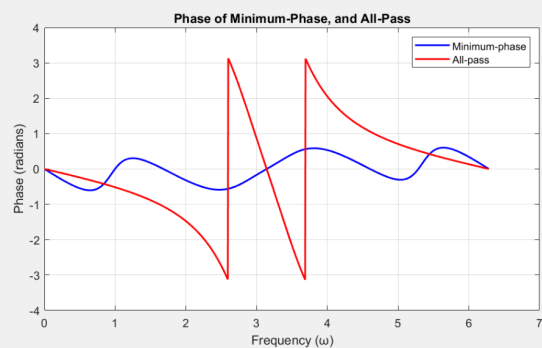
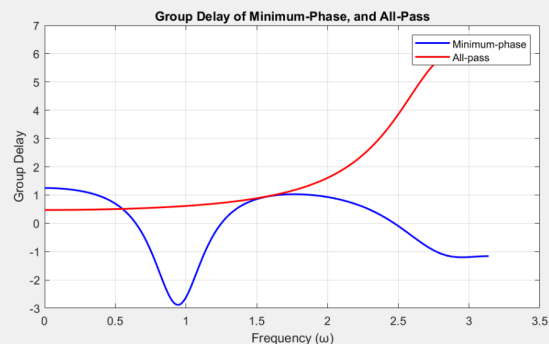
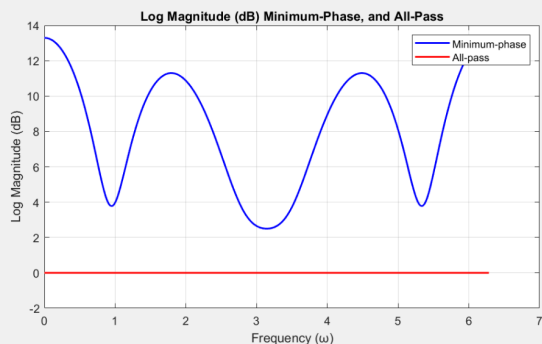
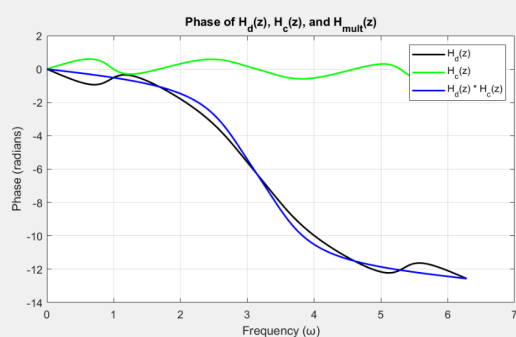
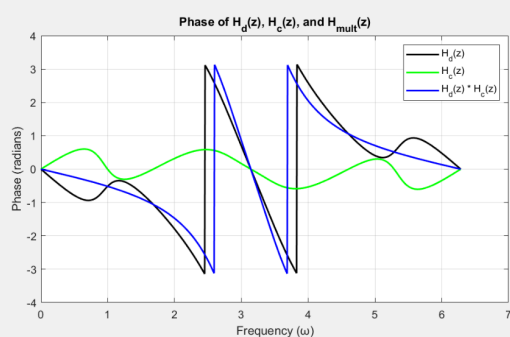
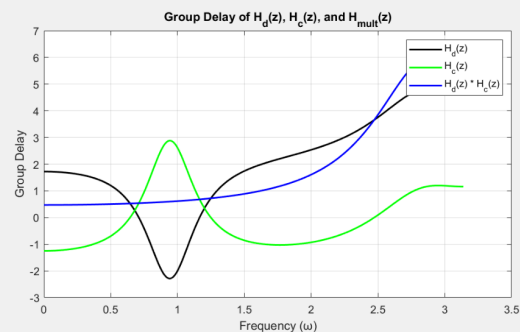
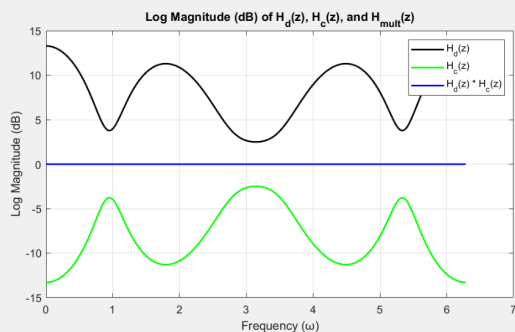
If you look at my bottom right graphs you will see the unwrapped phases. Basic visual inspection shows that as the unwrapped phase increases or decreases at a higher or lower rate, the group delay is changing accordingly to how a derivative would. It is very easy to see it is reflecting the change in the unwrapped phase. Later, you will see my results for the function given to us follows the same idea that the group delay is the negative derivative of the unwrapped phase.

Despite all of that, if you add ‘whole’ to the group delay, it has the property of constraining the frequency input to equi-spaced points all the unit circle. Essentially, it keeps the phase wrapped. Doing this provided the following result for the minimum-phase and all-pass group delays which are exactly the same as the book.

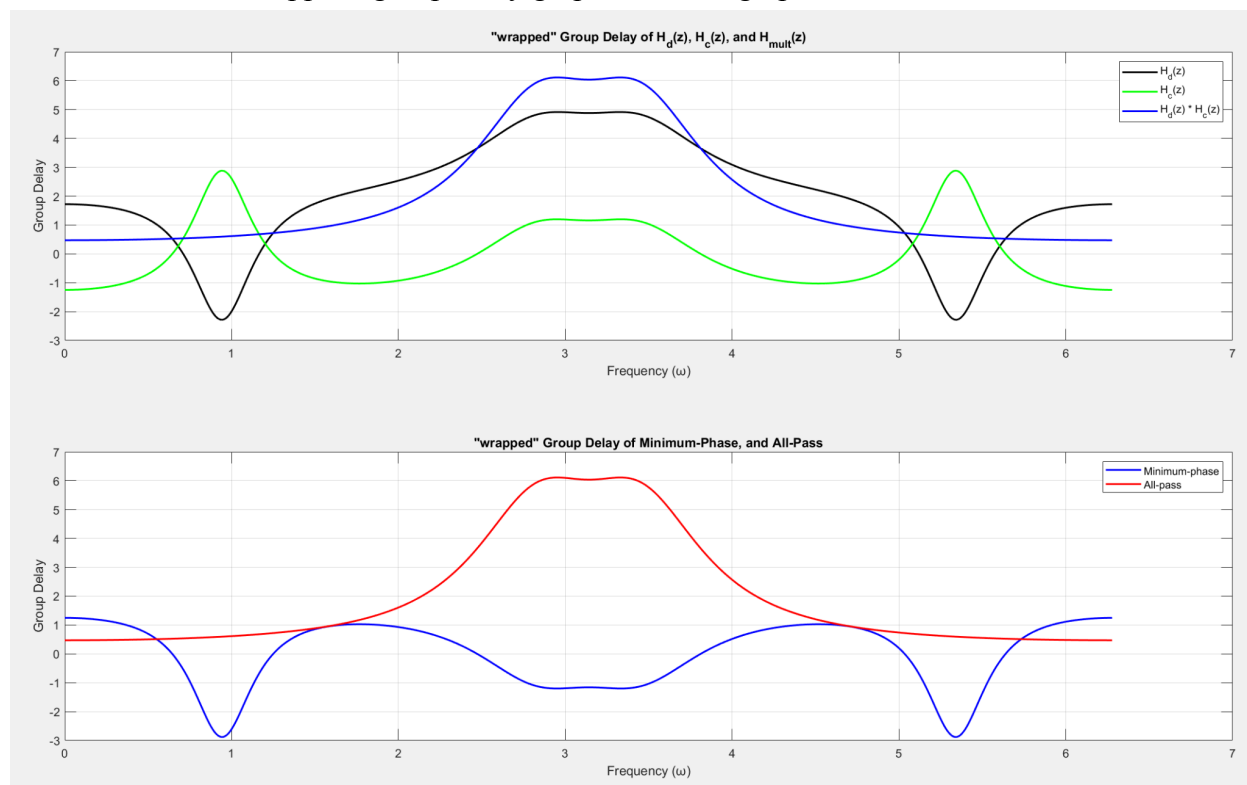


Results – Project

The results from my script are very promising: at the very least they are consistent. In the first 4 graph plot, the top left graph shows the log magnitude of the original distorted signal, the compensating filter, and the result of their convolution (multiplication in frequency domain). What we can see is that in terms of magnitude the compensating filter is exactly opposite that of the distorted signal and their convolution produces a unity magnitude output of the system—i.e. All frequencies are passed equally, i.e. an all-pass filter. The bottom left graph displays the wrapped phase of the previously mentioned three functions. We see discontinuities in the distorted signal phase and the overall system’s response due to this being the wrapped phase. This shape is similar to that from the book.



And here are the “wrapped” group delay graphs for each graph:



We can see in these spikes where there are discontinuities in the graphs.